SAMSUNG

Samsung Innovation Campus

Coding and Programming Course

Together for Tomorrow!
Enabling People
Education for Future Generations

Chapter 8.

Data Analysis and Visualization - Mini Project

C&P Course

Chapter Description

Chapter objectives

- ✔ Be able to be confident by making the same results with the data analysis and visualization as experts make in the field only with what we have learned through the course, which is the core goal of the mini-project.
- ✔ Be able to understand that data analysis skills using Python are essential skills in the real field by solving real-world cases through the mini-project.
- ✔ Be able to practice in the field of financial accounting through financial data analysis tasks.
- ✔ Be able to practice solving various real-world social problems through public data analysis tasks.

Chapter contents

- ✔ Unit 39. Financial Data Analysis Mini Project
- ✔ Unit 40. Global Corona Pandemic Analysis Mini Project

Unit 39.

Financial Data Analysis Mini Project

Mission

Financial Data Analysis Mini Project

- With the rapid increase in computing speed, mankind has been able to organize and analyze large amounts of data that have never been experienced before at incredibly high speed.
- Based on that, it is possible for computers to be learned and be developed to the current status of artificial intelligence.
- Even if you do not become a software engineer, you can solve many practical problems that you may see in the real world through the data processing skills you have learned so far using Python and especially Pandas.
- Pandas, a core library of data analysis applied in various fields, was originally developed to analyze and organize financial data.
- The core of data analysis is financial data analysis. Through the practice of acquiring and processing financial data of various factors accumulated over a period of time, you can be sufficiently prepared for most data types that you will see in real-world work.
- Financial data analysis is the most effective practice in data analysis.

Financial Data Analysis Mini Project

- Now, let's use all the skills we have learned so far to solve the following tasks.

  - Obtaining financial data from remote data repositories
  - Visualizing stock price data based on time series data
  - Visualizing trading volume data based on time series data
  - Measuring simple daily rate
  - Calculating simple daily cumulative rate of return
  - Calculating the rate of return by moving the period by month
  - Calculating moving average
  - Analyzing the correlation between each financial data factor Calculating stock price volatility

Financial Data Analysis Mini Project

- The data material used for data analysis is stock data that track various sectors and economic conditions. It is not simply analyzing the stock price of a specific company. It was chosen because there is no better target data to understand each economic situation as a real economy. In addition, the target data is not for companies, raw materials, bonds, or in a specific country, but for the price of agricultural products around the world.
- Since the US market has a very large impact on the real economy of the world, it was selected as a good practice with meaningful data.

Ticker name

- SPY, which tracks the leading US index S&P 500
- IYW, which tracks U.S. technology company market capitalization index
- VT, which invests in companies around the world
- DBA, which tracks the supply and demand of agricultural products and prices
- US Bond Rate TLT
- PDBC, which tracks the supply and demand of other raw materials and prices
- Gold IAU

Let’s code

Step 1

- Let’s prepare for data acquisition and create data acquisition functions.

```
1 import pandas as pd
2 import numpy as np
3 import datetime
4 import matplotlib.pyplot as plt
5 import yfinance as yf
6
7 from datetime import date, datetime, time, timezone
```

Step 1

- A function that obtains the stock price for a given stock when the ticker code is input

```
1  def get_stock_data(ticker,start,end):
2      data = yf.download(tickers=[ticker], start=start, end=end)
3      data.insert(0, "Ticker" , ticker)
4      return data
```

Line 2~3

- 2: Read the stock data from Yahoo Finance.
- 3: Create a column name called Ticker at index 0 of the data frame called data created above, and put the ticker name for the data element.

Step 1

- To test whether data acquisition works properly, enter Disney's ticker code to test the function.
- A ticker search can be easily done through Yahoo Finance or investing.com

```
1 ticker = 'DIS'
2 start = datetime (2020, 1,1)
3 end = datetime.today()
```

Line 1~3

- 1: Tiker code is obtained through search. Search for the ticker code of the company you are interested in and use it.
- 2: Specify the start date as a timestamp.
- 3: Specify to acquire data up to today. It can be specified as a specific date.

Step 1

```
1 d = get_stock_data(ticker,start,end)
2 d.head()
```

```
[**********100%%**********]  1 of 1 completed
```

Date	Ticker	Open	High	Low	Close	Adj Close	Volume
2020-01-02	DIS	145.289993	148.199997	145.100006	148.199997	148.199997	9502100
2020-01-03	DIS	146.399994	147.899994	146.050003	146.500000	146.500000	7320200
2020-01-06	DIS	145.539993	146.029999	144.309998	145.649994	145.649994	8262500
2020-01-07	DIS	145.990005	146.869995	145.419998	145.699997	145.699997	6906500
2020-01-08	DIS	145.490005	146.130005	144.820007	145.399994	145.399994	6984200



Line 1~2

- 1: Store the high price, low price, open price, close price, trading volume, and modified closing price data for the ticker in the data frame called d.
- 2: Check the data to see if the function works properly.

Step 1

- Since the analysis will be based on only the closing price for each ticker, pivot in the required form.

```
1 d=d.reset_index().pivot(index="Date", columns="Ticker", values="Close")
2 d.head()
```

Ticker	DIS
Date	
2020-01-02	148.199997
2020-01-03	146.500000
2020-01-06	145.649994
2020-01-07	145.699997
2020-01-08	145.399994

Step 2

- Create a data frame required for analysis using the created function.
- The name of the data frame is the name of each ticker: SPY / IYW / VT / DBA / TLT / PDBC / IAU

```
SPY = get_stock_data("SPY",start,end)
IYW = get_stock_data("IYW",start,end)
VT = get_stock_data("VT",start,end)
DBA = get_stock_data("DBA",start,end)
TLT = get_stock_data("TLT",start,end)
PDBC = get_stock_data("PDBC",start,end)
IAU = get_stock_data("IAU",start,end)
```

Step 2

```
1 SPY.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 914 entries, 2020-01-02 to 2023-08-18
Data columns (total 7 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Ticker     914 non-null    object
 1   Open       914 non-null    float64
 2   High       914 non-null    float64
 3   Low        914 non-null    float64
 4   Close      914 non-null    float64
 5   Adj Close  914 non-null    float64
 6   Volume     914 non-null    int64
dtypes: float64(5), int64(1), object(1)
memory usage: 57.1+ KB
```



Line 1

- It is a DatetimeIndex and there is no NaN.

Step 2

```
1 # Execute pivoting per data frame
2 SPY=SPY.reset_index().pivot(index="Date", columns="Ticker", values="Close")
3 IYW=IYW.reset_index().pivot(index="Date", columns="Ticker", values="Close")
4 VT=VT.reset_index().pivot(index="Date", columns="Ticker", values="Close")
5 DBA=DBA.reset_index().pivot(index="Date", columns="Ticker", values="Close")
6 TLT=TLT.reset_index().pivot(index="Date", columns="Ticker", values="Close")
7 PDBC=PDBC.reset_index().pivot(index="Date", columns="Ticker", values="Close")
8 IAU=IAU.reset_index().pivot(index="Date", columns="Ticker", values="Close")
```



Line 1

- Execute pivoting per data frame.

Step 2

- Each created data frame is combined into one data frame for efficient analysis.
- At this time, if the configuration and properties of the data frames to be merged are the same, data consistency can be maintained regardless of the direction of the row or column. Remember that this is something you must check before merging data frames.
- Each data frame we're going to merge now has the same index, the same column, and the same type of data elements. Therefore, we use a function concat() to concatenate while maintaining the shape of the existing data

```
pandas.concat(objs, axis=0, join='outer', ignore_index=False, keys=None, levels=None, names=None, verify_integrity=False, sort=False, copy=True)
```

- https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.concat.html

Step 2

```
1 stock=pd.concat([SPY, IYW, VT, DBA, TLT, PDBC, IAU],
2                 axis=1,
3                 join='outer')
4 stock.head()
```

Ticker	SPY	IYW	VT	DBA	TLT	PDBC	IAU
Date							
2020-01-02	324.869995	59.355000	81.809998	16.500000	137.009995	16.639999	29.219999
2020-01-03	322.410004	58.762501	81.070000	16.309999	139.119995	16.780001	29.620001
2020-01-06	323.640015	59.125000	81.370003	16.350000	138.330002	16.799999	29.920000
2020-01-07	322.730011	59.147499	81.120003	16.389999	137.649994	16.770000	30.040001
2020-01-08	324.450012	59.737499	81.449997	16.309999	136.740005	16.540001	29.820000

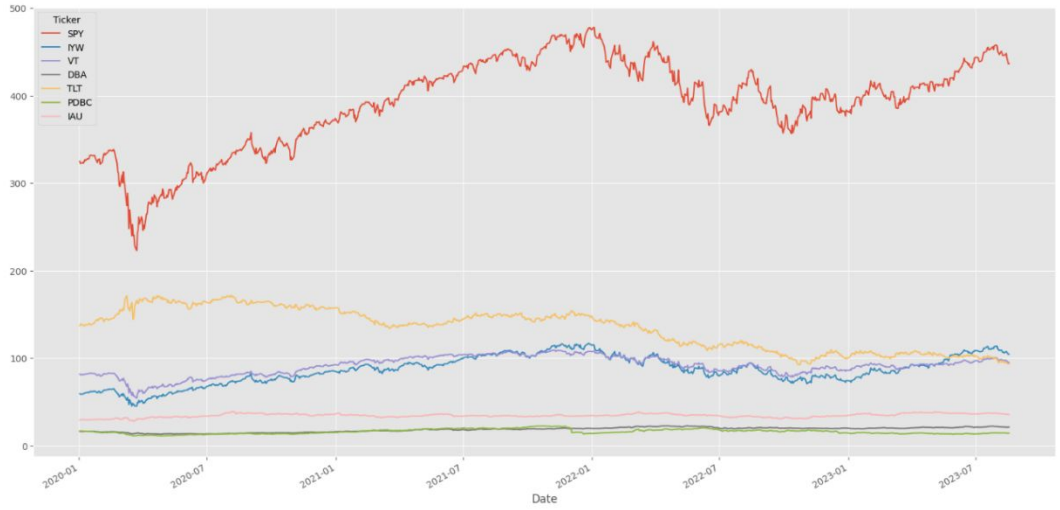
Line 1~3

- 1: List the data frames to be concatenated.
- 2: The default option is axis=0. Each data frame is concatenated vertically.
- 3: If the column name is inner, the standard is the intersection of each data frame. The default value is outer.

Step 3

- If you draw a graph based on the closing price data on a time series basis (date), you can check the stock price movement. By overlapping the graphs of each ticker, you can see the approximate correlation.

```
1  plt.style.use('ggplot')
2  stock.plot(figsize = (20, 10))
3  plt.show()
```



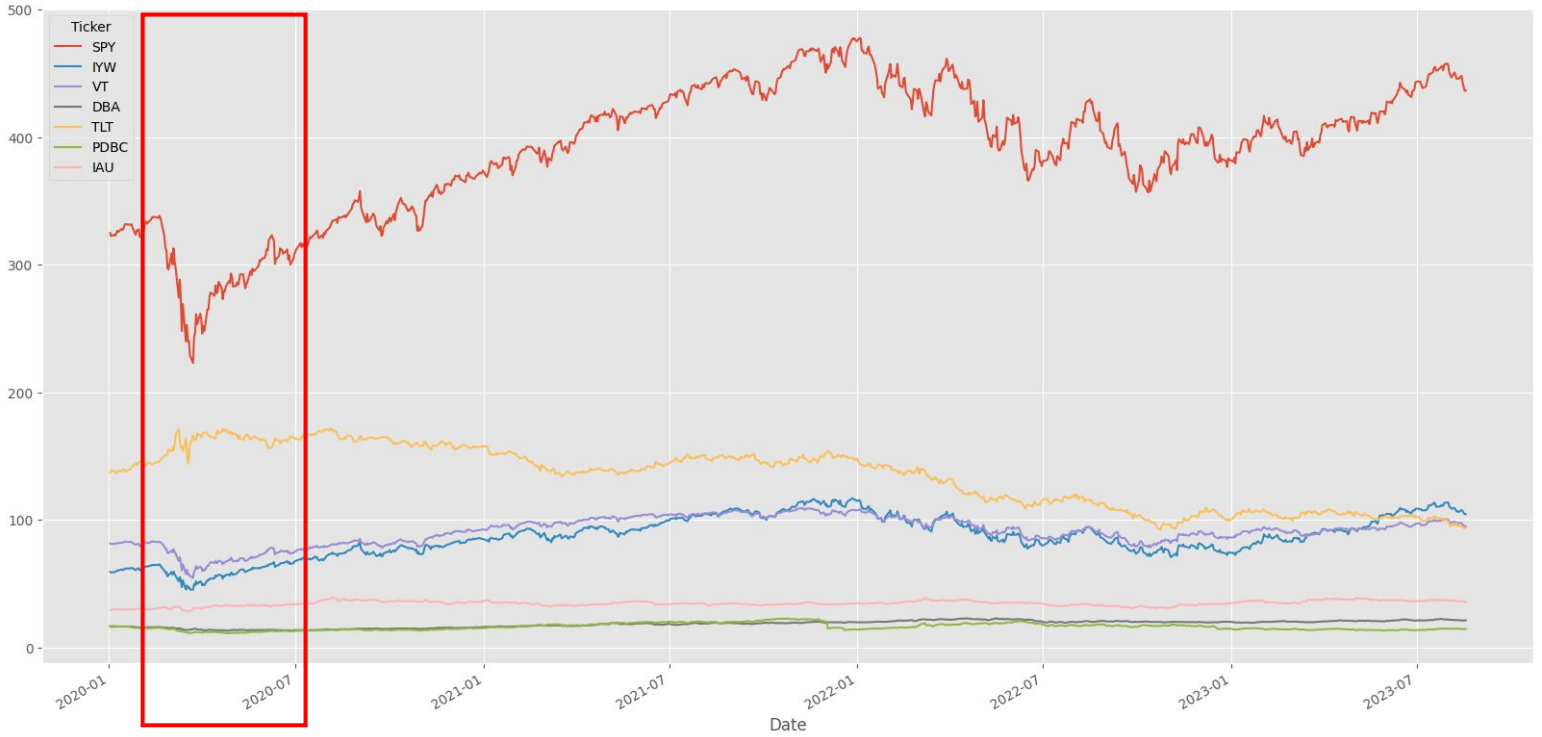
Line 1

- Specify the graph style.

Step 3

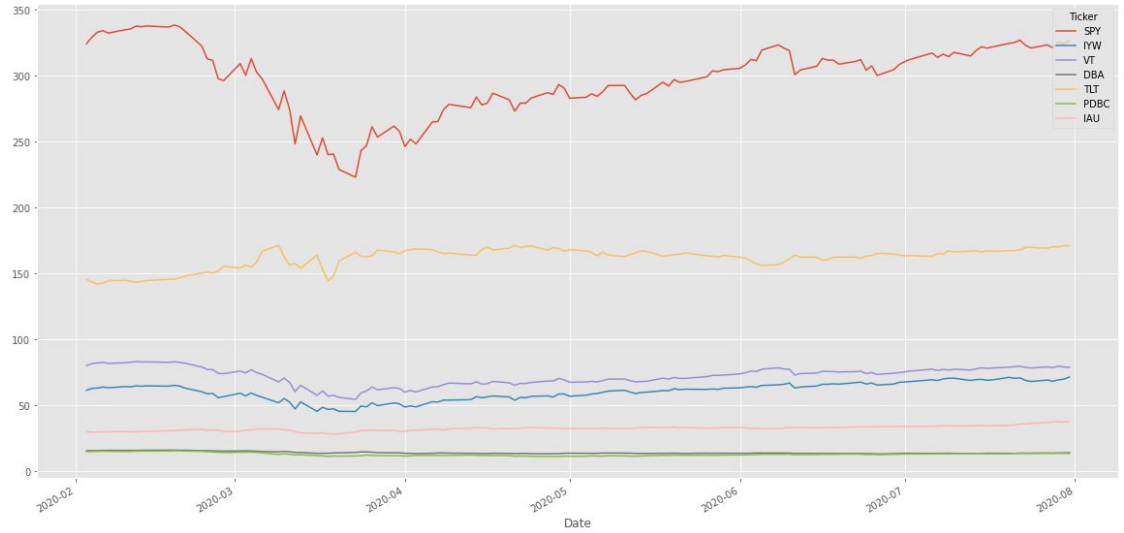
- If you see the graph below, there is a section with serious price fluctuations. This is the period when Corona started. Let's check the data by slicing using the time series data index only for this period.

```
1 covid=stock['2020-2-1':'2020-7-31']
```



Step 3

```
1 plt.style.use('ggplot')
2 covid.plot(figsize = (20, 10))
3 plt.show()
```



Line 1~3

- 1: Specify the graph style.
- 3: There are factors that have a large impact on price fluctuations in the aftermath of the corona shock, while there are factors that rise to that point or have little effect.

Step 3

- Let's separate the relevant period factors to check the graph.

```
1 x = covid.index
2 s_y = covid[['SPY']]
3 i_y = covid[['IAU']]
4 d_y = covid[['DBA']]
5 t_y = covid[['TLT']]
```



Line 2

- Select three pieces of data with different personalities and compare them.

Step 3

- Let's compare how assets with different personalities react during times of great shock to economic conditions like the coronavirus.
- You can see that the general stock price declines sharply and then gradually recovers, but gold and bonds are the opposite. In particular, in the case of bonds, you can see that the trend is opposite to the stock price.

```
1  import matplotlib.pyplot as plt
2
3  fig, axs = plt.subplots(1, 3, figsize=(15, 5))
4  axs[0].plot(x, s_y)
5  axs[1].plot(x, i_y)
6  axs[2].plot(x, t_y)
7
8
9  fig.suptitle('Covid 19')
```

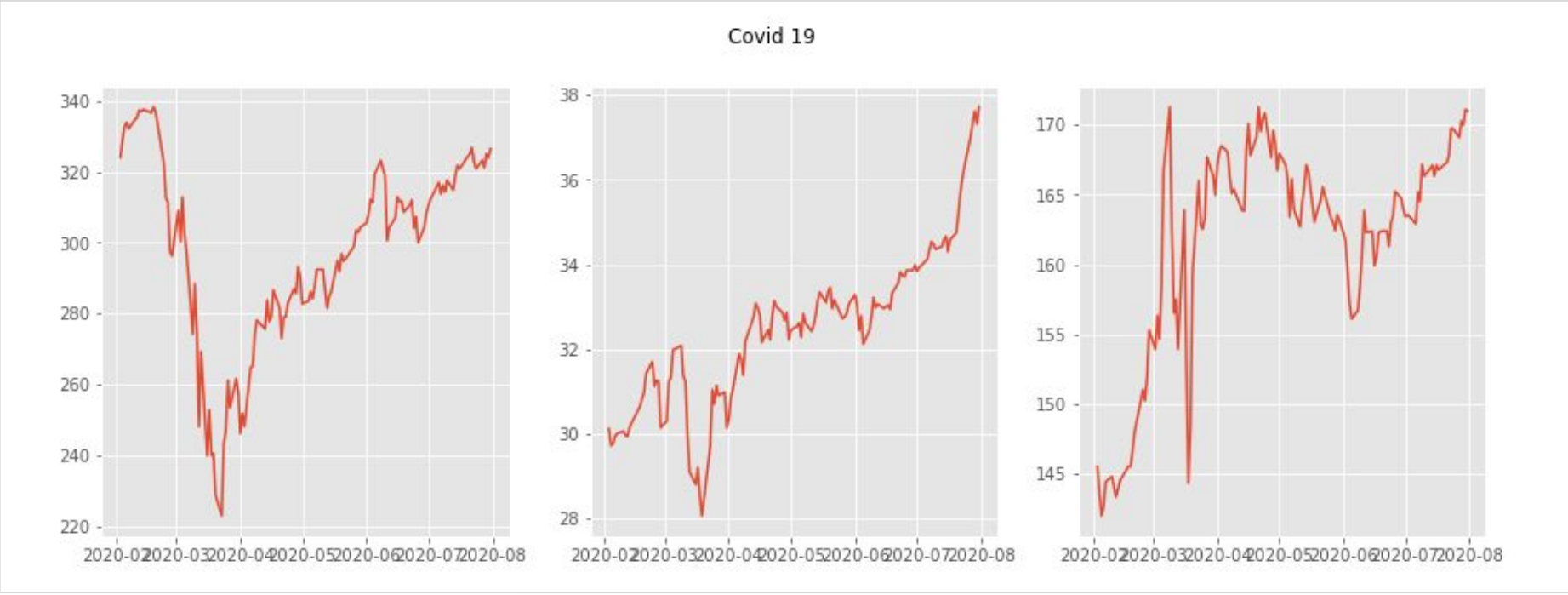
```
Text(0.5, 0.98, 'Covid 19')
```

Line 4~6

- 4: SPY
- 5: IAU
- 6: TLT

Step 3

- These graphs are the result of the previous code.



Step 4

- Let's visualize the trading volume data in the form of a bar graph for a specific ticker.

```
import pandas as pd
import numpy as np
import datetime
import matplotlib.pyplot as plt
import yfinance as yf

from datetime import date, datetime, time, timezone
```

```
def get_stock_data(ticker,start,end):
    data = yf.download(tickers=[ticker], start=start, end=end)
    data.insert(0, "Ticker" , ticker)
    return data
```

Step 4

```
1 ticker = 'PDBC'
2 start = datetime(2020,1,1)
3 end = datetime.today()
```

Line 1~3

- Statistics on raw materials
- Specify the start date with a timestamp.
- Specify to acquire data up to today.

Step 4

```
1 df = get_stock_data(ticker, start, end)
2 df.head()
```

```
[**********100%%**********]  1 of 1 completed
```

Date	Ticker	Open	High	Low	Close	Adj Close	Volume
2020-01-02	PDBC	16.570000	16.670000	16.520000	16.639999	9.537181	4004600
2020-01-03	PDBC	16.809999	16.840000	16.709999	16.780001	9.617422	760200
2020-01-06	PDBC	16.900000	16.910000	16.770000	16.799999	9.628882	1608700
2020-01-07	PDBC	16.750000	16.799999	16.716999	16.770000	9.611690	1723200
2020-01-08	PDBC	16.770000	16.770000	16.363001	16.540001	9.479866	923300

Line 1

- In a data frame called df, the high price, low price, open price, closing price, trading volume, and modified closing price data for the corresponding ticker are stored.

Step 4

```
1 df.drop(['Ticker', 'High', 'Low', 'Open', "Close", "Adj Close"], axis=1, inplace=True)
```

Line 1

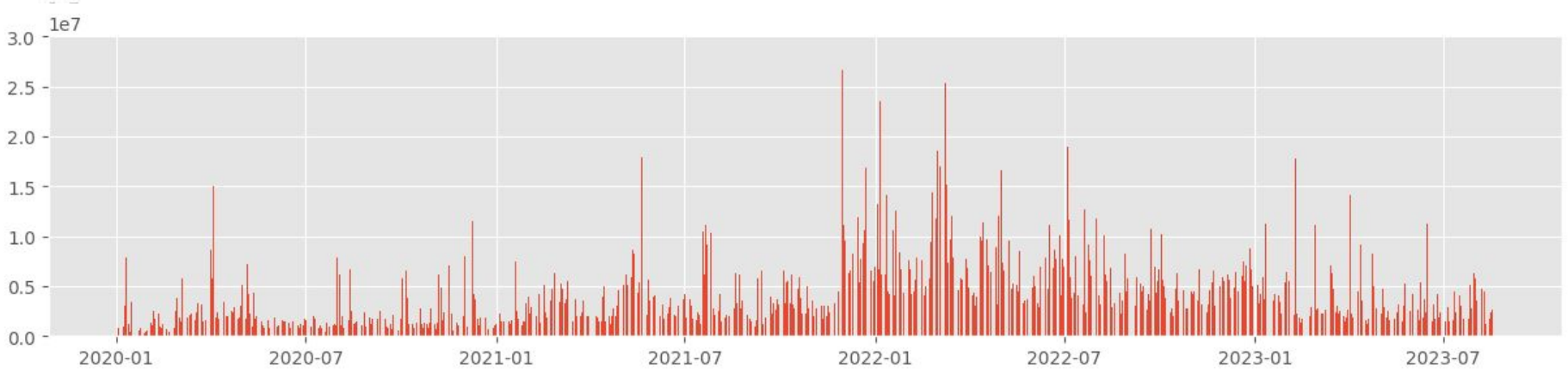
- Delete all except the volume column.

```
1 df.head()
```

Date	Volume
2020-01-02	4004600
2020-01-03	760200
2020-01-06	1608700
2020-01-07	1723200
2020-01-08	923300

Step 4

```
1 x = df.index
2 y = df['Volume']
3 plt.figure(figsize = (15,3))
4 plt.bar(x, y)
5 plt.show()
```



Step 5

- Using matplotlib's subplot2grid, the graph of the closing price is visualized in the upper layout and the trading volume in the same period is visualized in the lower layout.

ax = subplot2grid((nrows, ncols), (row, col), rowspan, colspan)

- https://matplotlib.org/stable/gallery/userdemo/demo_gridspec01.html#sphx-glr-gallery-userdemo-demo-gridspec01-py

```
1 ticker = 'PDBC'
2 start = datetime(2020,1,1)
3 end = datetime.today()
```

Line 1~3

- Statistics on raw materials
- Specify the start date with a timestamp.
- Specify to acquire data up to today.

```
1 df = get_stock_data(ticker, start, end)
```

Step 5

ax = subplot2grid((nrows, ncols), (row, col), rowspan, colspan)

```
1  fig = plt.figure(figsize=(12, 8))
2  top_grid = plt.subplot2grid((4,4), (0,0), rowspan=3, colspan=4)
3  bottom_grid = plt.subplot2grid((4,4), (3,0), rowspan=1, colspan=4)
4
5  top_grid.plot(df.index, df['Close'], label='Close')
6  bottom_grid.plot(df.index, df['Volume'], label='Volume')
7
8  plt.tight_layout()
9
10 plt.legend()
11 plt.show()
```

Line 5

- Show closing price data on the upper grid.

Step 5

ax = subplot2grid((nrows, ncols), (row, col), rowspan, colspan)

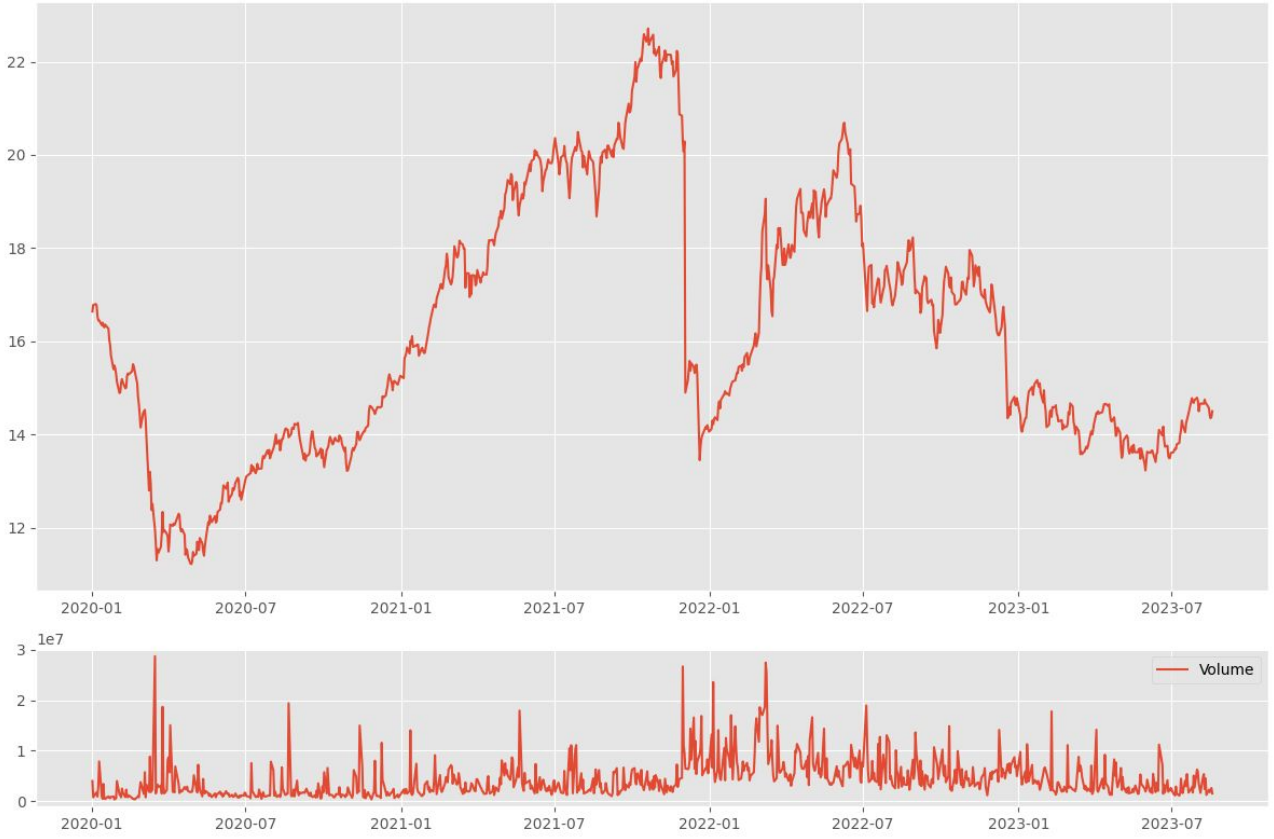
```
1  fig = plt.figure(figsize=(12, 8))
2  top_grid = plt.subplot2grid((4,4), (0,0), rowspan=3, colspan=4)
3  bottom_grid = plt.subplot2grid((4,4), (3,0), rowspan=1, colspan=4)
4
5  top_grid.plot(df.index, df['Close'], label='Close')
6  bottom_grid.plot(df.index, df['Volume'], label='Volume')
7
8  plt.tight_layout()
9
10 plt.legend()
11 plt.show()
```

Line 6~8

- Show volume data on the lower grid.
- A function that allows subplots to be printed at the maximum size in the figure

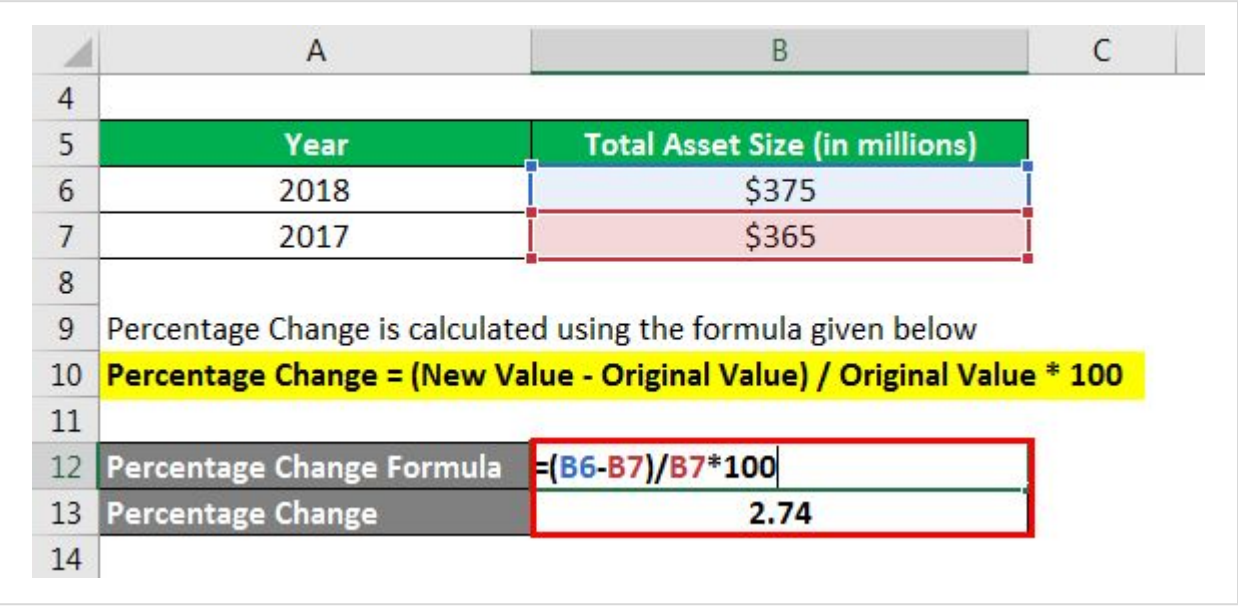
Step 5

ax = subplot2grid((nrows, ncols), (row, col), rowspan, colspan)



Step 6

- **We will use the pandas.Series.shift() method to calculate the daily percentage change.**
- https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.shift.html?highlight=shift#pandas.Series.shift
- Comparing the current data with the data at a specific point in time to calculate the percentage change is also a common task in daily work.
- If you solve this with Excel, you can do it as shown below. If you see the image, you can understand why the shift() method is useful.



	A	B	C
4			
5	Year	Total Asset Size (in millions)	
6	2018	$375	
7	2017	$365	
8			
9	Percentage Change is calculated using the formula given below		
10	**Percentage Change = (New Value - Original Value) / Original Value * 100**		
11			
12	Percentage Change Formula	=(B6-B7)/B7*100	
13	Percentage Change	2.74	
14			

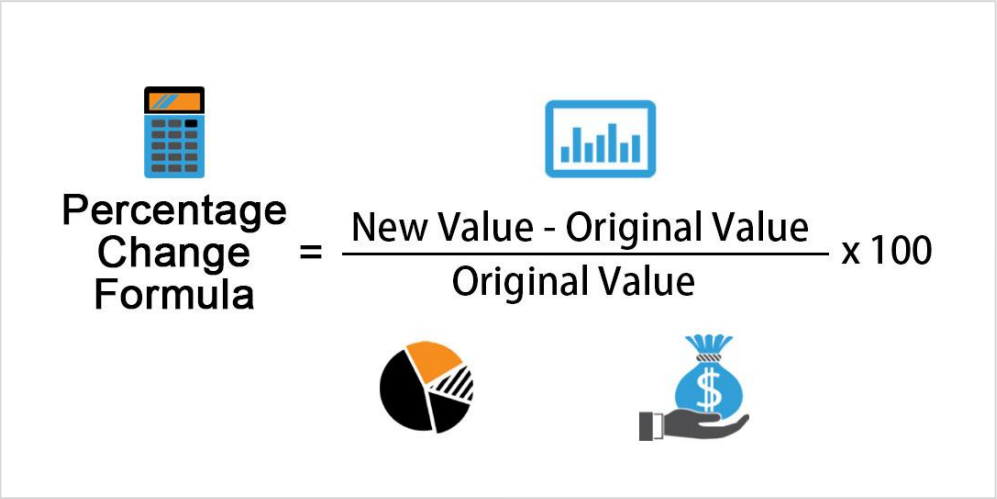
Step 6

pandas.Series.shift()

- The formula for calculating the daily percentage change based on the closing price is simple

Today's daily percentage change = ((Today's closing price - Yesterday's closing price) / Yesterday's closing price) * 100

Daily Percentage Change = (New Value – Original Value) / Original Value * 100



https://www.educba.com/percentage-change-formul a/

Step 6

pandas.Series.shift()

```
1 stock.head()
```

Ticker	SPY	IYW	VT	DBA	TLT	PDBC	IAU
Date							
2020-01-02	324.869995	59.355000	81.809998	16.500000	137.009995	16.639999	29.219999
2020-01-03	322.410004	58.762501	81.070000	16.309999	139.119995	16.780001	29.620001
2020-01-06	323.640015	59.125000	81.370003	16.350000	138.330002	16.799999	29.920000
2020-01-07	322.730011	59.147499	81.120003	16.389999	137.649994	16.770000	30.040001
2020-01-08	324.450012	59.737499	81.449997	16.309999	136.740005	16.540001	29.820000

Step 6

pandas.Series.shift()

```
1 stock['SPY']
```

```
Date
2020-01-02    324.869995
2020-01-03    322.410004
2020-01-06    323.640015
2020-01-07    322.730011
2020-01-08    324.450012
                 ...
2023-08-14    448.109985
2023-08-15    442.890015
2023-08-16    439.640015
2023-08-17    436.290009
2023-08-18    436.500000
Name: SPY, Length: 914, dtype: float64
```

Step 6

pandas.Series.shift()

```
1 stock['SPY'].shift(1)
```

```
Date
2020-01-02           NaN
2020-01-03    324.869995
2020-01-06    322.410004
2020-01-07    323.640015
2020-01-08    322.730011
                 ...
2023-08-14    445.649994
2023-08-15    448.109985
2023-08-16    442.890015
2023-08-17    439.640015
2023-08-18    436.290009
Name: SPY, Length: 914, dtype: float64
```

Line 1

- Shift one day to get the previous day's closing price.

Step 6

pandas.Series.shift()

```
1 spy_dayily_pc=(stock['SPY']/stock['SPY'].shift(1)-1)*100
```

```
1 spy_dayily_pc
```

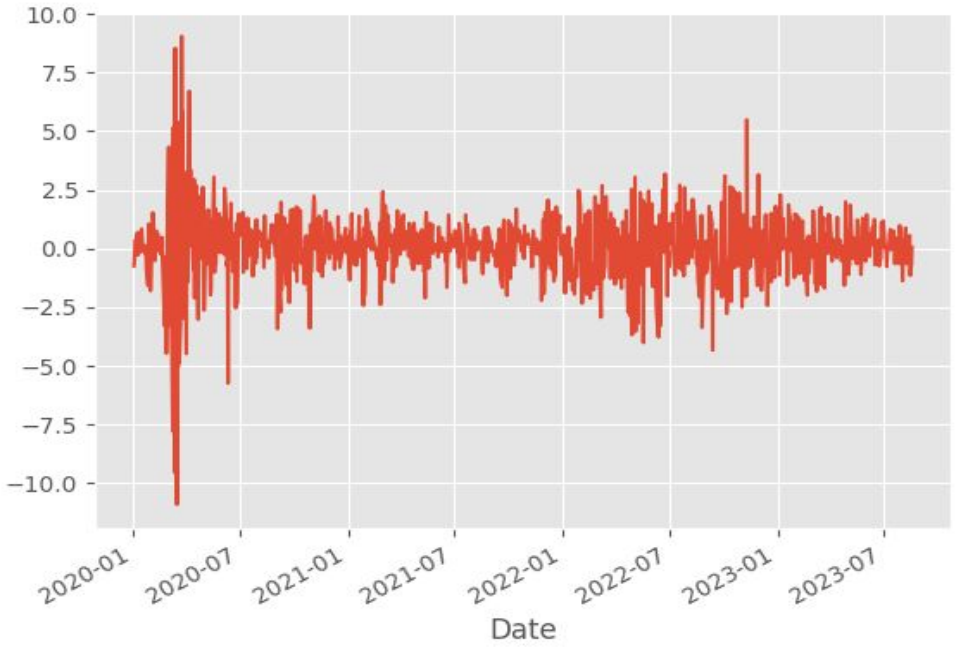
```
Date
2020-01-02         NaN
2020-01-03   -0.757223
2020-01-06    0.381505
2020-01-07   -0.281178
2020-01-08    0.532954
                ...
2023-08-14    0.552001
2023-08-15   -1.164886
2023-08-16   -0.733816
2023-08-17   -0.761988
2023-08-18    0.048131
Name: SPY, Length: 914, dtype: float64
```

Step 6

pandas.Series.shift()

```
1 spy_dayily_pc.plot()
```

```
<Axes: xlabel='Date'>
```



Step 6

- A histogram is a graph showing the frequency distribution.
- It represents the frequency of data values by time.
- At this time, the number of sections is used by setting the parameter bins value of the hist() function.
- For reference, the default value of hist() is 10. The shape of the graph changes depending on the bins. You should set the bins value by carefully checking the characteristics of the data.

Step 6

pandas.DataFrame.hist( )

- https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.hist.html?highlight=hist#pandas.DataFrame.hist

```
1 spy_dayily_pc=(stock['SPY']/stock['SPY'].shift(1)-1)*100
```

Line 1

- Don't confuse the new formula. It's just written out to help you understand.

```
1 spy_dayily_pc.iloc[0] =0
```

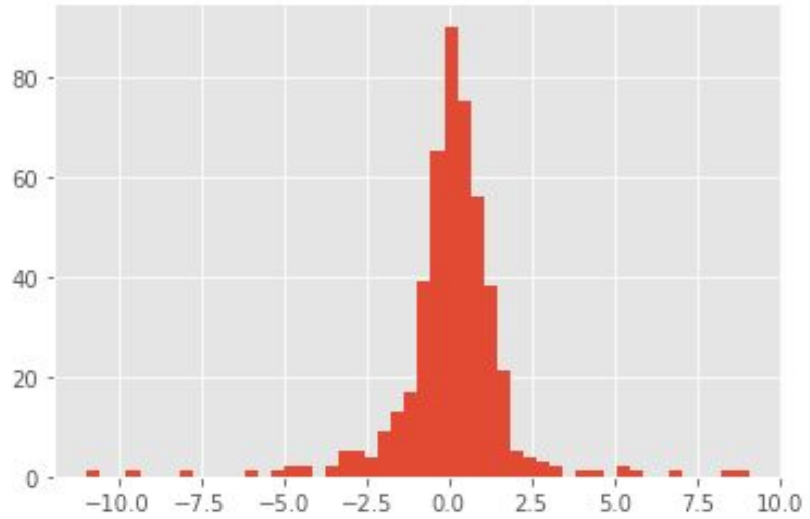
Line 1

- Since the first value is missing data, it is replaced with 0.

Step 6

pandas.DataFrame.hist( )

```
1 plt.hist(spy_dayily_pc, bins = 50)
2 plt.show()
```



Line 1

- The frequency is expressed by dividing the daily percentage change of the stock price into 50 sections.

Step 7

- Let’s create a new data frame that calculates daily stock price changes for all tickers, calculate daily cumulative returns, and analyze the correlation.

```
1 stock_dayily_pc=(stock-stock.shift(1))/stock.shift(1) * 100
```

```
1 stock_dayily_pc.head()
```

Ticker	SPY	IYW	VT	DBA	TLT	PDBC	IAU
Date							
2020-01-02	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2020-01-03	-0.757223	-0.998229	-0.904532	-1.151518	1.540034	0.841354	1.368931
2020-01-06	0.381505	0.616889	0.370054	0.245254	-0.567850	0.119181	1.012827
2020-01-07	-0.281178	0.038053	-0.307239	0.244642	-0.491584	-0.178564	0.401072
2020-01-08	0.532954	0.997507	0.406798	-0.488102	-0.661089	-1.371494	-0.732361

Step 7

- The formula for the simple daily cumulative return is also simple.
- It can be obtained by accumulating and multiplying the daily stock price change rate obtained above. We can use the .compsum() method.

```
1 stock_d_cr=stock_dayily_pc.cumsum()
```

Step 7

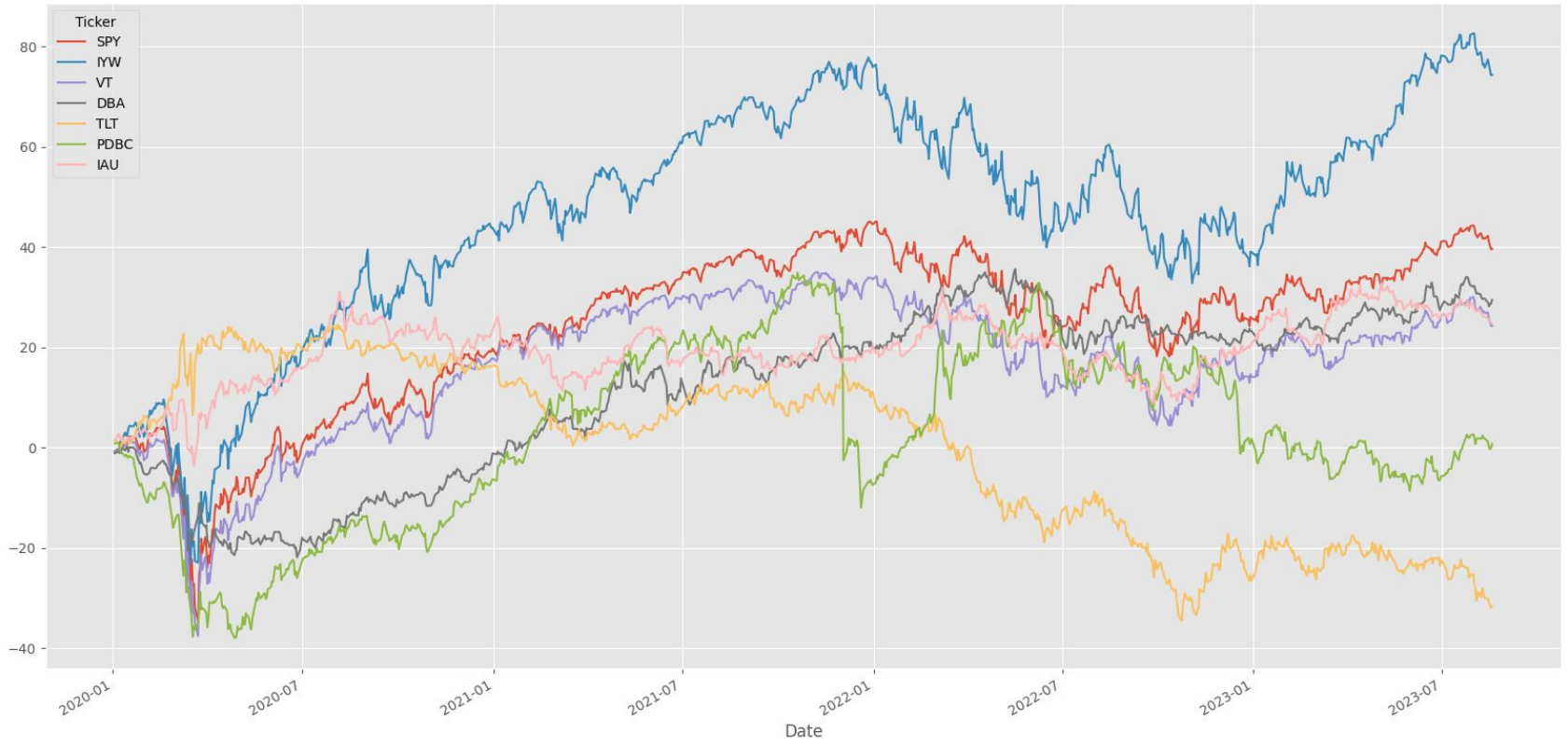
```
1 stock_d_cr
```

Ticker	SPY	IYW	VT	DBA	TLT	PDBC	IAU
Date							
2020-01-02	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2020-01-03	-0.757223	-0.998229	-0.904532	-1.151518	1.540034	0.841354	1.368931
2020-01-06	-0.375718	-0.381340	-0.534478	-0.906264	0.972184	0.960535	2.381757
2020-01-07	-0.656896	-0.343287	-0.841717	-0.661622	0.480600	0.781971	2.782830
2020-01-08	-0.123942	0.654220	-0.434919	-1.149724	-0.180489	-0.589523	2.050469
...	...	...	...	...	...	...	...
2023-08-14	42.207636	77.393050	26.908305	28.953835	-30.185181	1.180581	25.820268
2023-08-15	41.042750	76.391007	25.728938	28.153834	-30.794683	0.082435	25.515894
2023-08-16	40.308933	75.331965	24.971358	28.675659	-31.577095	-0.333938	24.960810
2023-08-17	39.546945	74.280530	24.312572	28.628466	-32.003346	-0.124881	24.793349
2023-08-18	39.595076	74.318823	24.249417	29.431110	-31.650184	0.709611	24.821300

Step 7

```
1 stock_d_cr.plot(figsize = (20, 10))
```

```
<Axes: xlabel='Date'>
```



Step 8

- The correlation coefficient refers to measuring the strength of the association between data as we learned earlier. The closer to 1.0, the stronger the relationship, and the closer to 0, the less the relationship. Let's analyze it using the learned .corr().

```
1 df_corr = stock_dayily_pc.corr()
```

```
1 df_corr
```

Ticker	SPY	IYW	VT	DBA	TLT	PDBC	IAU
Ticker							
SPY	1.000000	0.919866	0.976368	0.257414	-0.209502	0.299467	0.147197
IYW	0.919866	1.000000	0.889478	0.200897	-0.122592	0.228892	0.158590
VT	0.976368	0.889478	1.000000	0.285928	-0.202603	0.332105	0.190714
DBA	0.257414	0.200897	0.285928	1.000000	-0.151158	0.464230	0.193688
TLT	-0.209502	-0.122592	-0.202603	-0.151158	1.000000	-0.132000	0.274215
PDBC	0.299467	0.228892	0.332105	0.464230	-0.132000	1.000000	0.217698
IAU	0.147197	0.158590	0.190714	0.193688	0.274215	0.217698	1.000000

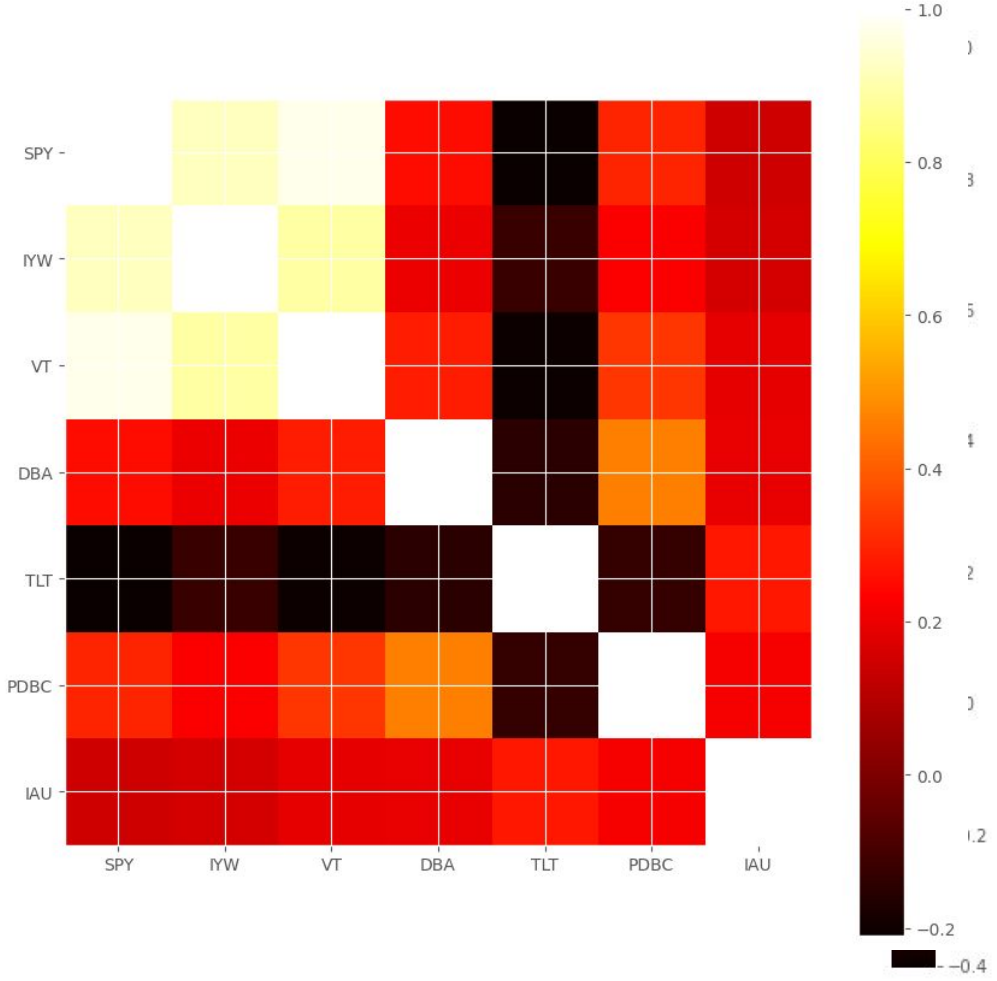
Step 8

- Let's visualize it as a heatmap.
- The darker the color, the lower the correlation, and the brighter the color, the higher the correlation. Let's check how each economic factor correlates in the real economy.

```
plt.imshow (df_corr, cmap ='hot', interpolation ='none')
plt.colorbar()
plt.xticks(range(len(df_corr)),df_corr.columns)
plt.yticks(range(len(df_corr)),df_corr.columns)

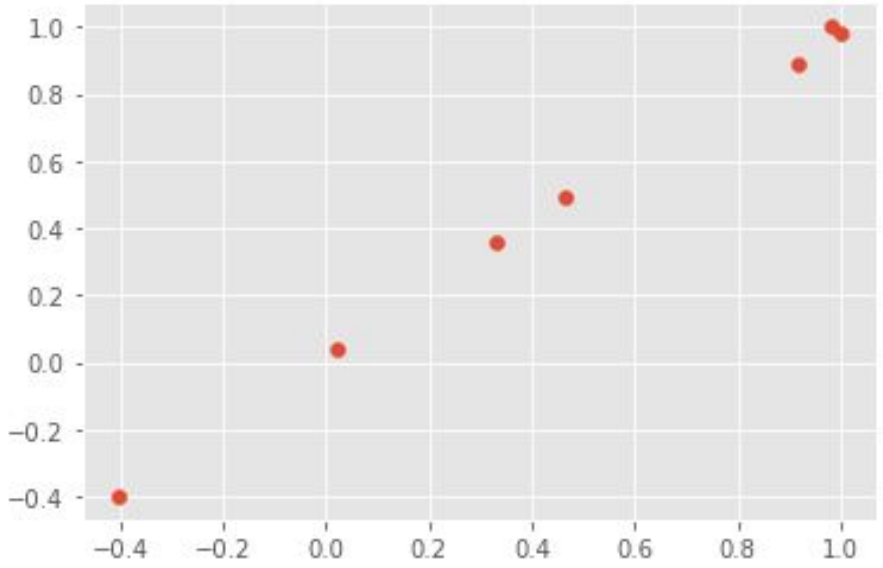
plt.gcf().set_size_inches(10,10)
```

Step 8


SPY
IYW
VT
DBA
TLT
PDBC
IAU
SPY
IYW
VT
DBA
TLT
PDBC
IAU
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

Step 8

```
1 plt.scatter(df_corr.SPY,df_corr.VT)
2 plt.show()
```



Step 8

Ex You can see that US bonds and general stocks are completely uncorrelated by economic conditions.

- When the economy is good, the company's performance improves, and the stock price rises. Conversely, when the economy is bad, the value of bonds rises. It is now possible to be verified through actual data analysis rather than theory.

- If you think once more by applying the correlation coefficient in practice, there is a complementary effect for each asset. You can create a strategy to mitigate the risk.
- A correlation coefficient close to 1 means that the value rises when it rises and falls when it falls. It means that there is no risk mitigation effect on each other. If the correlation coefficient is close to 0, it means that there is no relationship between the value rises and falls, and on the contrary, the risk mitigation effect is large.
- For example, it can be used in making a marketing plan in the real world. As a data-based decision-making tool, it can be used for analyzing the correlation between sales timing and sales by product, setting a product launch time, and building company portfolios.

Step 9

- Let's track the change in stock price over a specific period of time. This technique is usually used to determine the risk rate of the stock by comparing the volatility of the entire market index as reference data.
- The purpose is to measure the amount of change in a specific period and compare it with stable reference data to determine the risk of the currently evaluated data.
- First, the stock price volatility can be obtained by calculating the standard deviation of the stock price volatility through the moving average. The biggest influence on this data is the period of time to be tracked. That is, the size of the window has a large effect.
- If the window is wide, representativeness will be blurred, and if it is too narrow, it will be close to the standard deviation. It is very difficult and important to set the size of the window to be measured.
- Not everything can be concluded through computer calculations. There are many moments when people have to make a decision based on understanding the context and the outcome to achieve.
- It is very important to understand that the results of the data can be different depending on the judgment of the learner, and the decision-making in the real world can be changed accordingly, rather than just learning the skills as a practical task using pandas.

Step 9

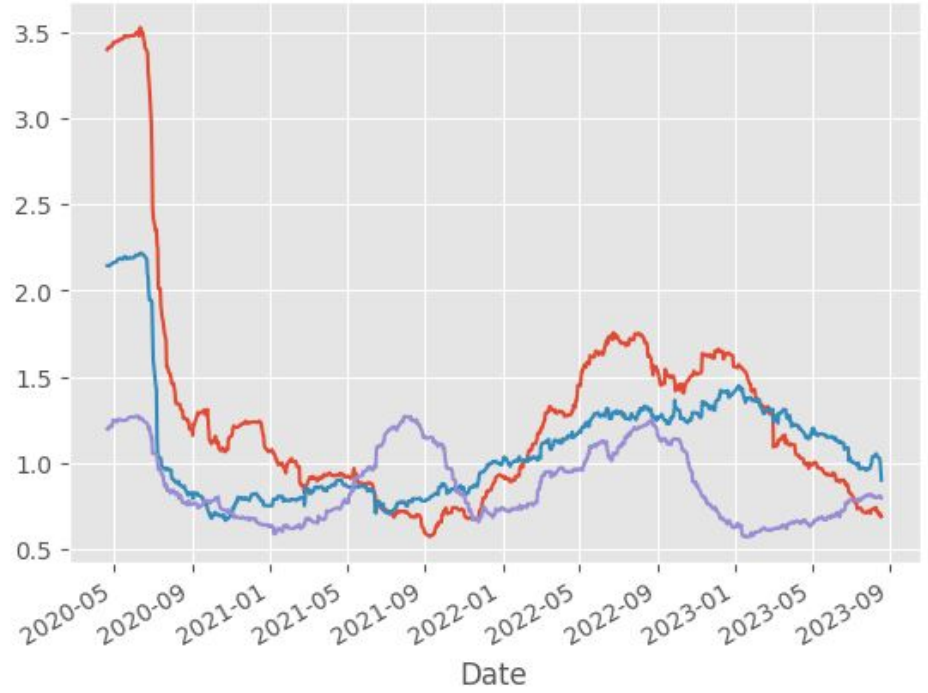
```
1 periods = 75
2
3 vol = stock_dayily_pc.rolling(window=periods).std()
4
5 vol
```

Ticker	SPY	IYW	VT	DBA	TLT	PDBC	IAU
Date							
2020-01-02	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2020-01-03	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2020-01-06	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2020-01-07	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2020-01-08	NaN	NaN	NaN	NaN	NaN	NaN	NaN
...	...	...	...	...	...	...	...
2023-08-14	0.709775	1.196046	0.744809	0.799927	1.028980	1.041705	0.699201
2023-08-15	0.691858	1.172083	0.735916	0.805150	1.025620	1.050092	0.699735
2023-08-16	0.692696	1.178928	0.739027	0.807106	1.007713	1.044457	0.701955
2023-08-17	0.699157	1.187409	0.743159	0.797413	0.946740	1.040431	0.700190
2023-08-18	0.685125	1.179022	0.730919	0.794489	0.897727	1.003728	0.659761

Step 9

```
1 vol["SPY"].plot()
2 vol["TLT"].plot()
3 vol["DBA"].plot()
```

```
<Axes: xlabel='Date'>
```



Unit 40.

Global Corona Pandemic Analysis Mini Project

Mission

Global Corona Pandemic Analysis Mini Project

- In the history of mankind, there have been several pandemic situations.
- Every time a new epidemic appears, mankind has been trying to find new vaccines and treatments. In each case, the most important role was to conduct reliable epidemiological investigations and transparently disclose the results.
- Mankind created new vaccines and treatments based on publicly released data and tried to find a way until an alternative is available.
- In the pandemic situation, various data analyses (e.g., How many people are currently infected? Where are the regions? Where are infected people currently decreasing? What is the age and living environment? Where the number of infected people is high? Is the number of infected people decreasing due to the effectiveness of the vaccine?) and data visualization for people to understand easier have made a lot of contributions.
- Through this mini-project, we are also trying to get descriptive statistics on which countries have the highest number of infections based on the coronavirus-related data available to the public.
- The result will display the cumulative infection level on a map of the world so that anyone can check it at a glance.
- We use the data from the [Coronavirus (COVID-19) Vaccinations] database collected in real-time by https://ourworldindata.org/
- We can use the data required for practice in **"./data//covid/covid-vaccination-doses-per-capita.csv".**
- For reference, we need to understand that there are some data are excluded because some countries do not disclose the data to the world, but we can still practice with the data provided.

Global Corona Pandemic Analysis Mini Project

Let’s code

Step 1

- Let’s prepare the data.

```
1 import numpy as np
2 import pandas as pd
3 import datetime
4 import matplotlib.pyplot as plt
5 from datetime import date, datetime, time, timezone
6
7 df = pd.read_csv("./data/covid/covid-vaccination-doses-per-capita.csv")
```



Line 7

- Enter the path of the downloaded file as a relative path.

Step 1

```
1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 77859 entries, 0 to 77858
Data columns (total 4 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   Entity                          77859 non-null  object
 1   Code                            77859 non-null  object
 2   Day                             77859 non-null  object
 3   total_vaccinations_per_hundred  77859 non-null  float64
dtypes: float64(1), object(3)
memory usage: 2.4+ MB
```



Line 1

- Check the technical summary of the data.

Step 1

```
1 df['Date'] = pd.to_datetime(df['Day'])
2 df.set_index('Date', inplace= True)
3 df.drop(['Day'], axis=1, inplace=True)
4
5 df.head()
```

Date	Entity	Code	total_vaccinations_per_hundred
2021-02-22	Afghanistan	AFG	0.00
2021-02-28	Afghanistan	AFG	0.02
2021-03-16	Afghanistan	AFG	0.13
2021-04-07	Afghanistan	AFG	0.29
2021-04-22	Afghanistan	AFG	0.58

Line 1, 2, 3, 5

- 1: Since Day is an Object data type, there is no need to change it to a character type, just convert it to datetime.
- 2: Change the index to the newly created column.
- 3: Delete unnecessary columns.
- 5: Check the data frame.

Step 1

```
1 len (df['Entity'].unique())
```

```
235
```

Line 1

- You can check the data from a total of 235 countries.

Step 2

- Let's create a group around the entity column and create a new data frame to store accumulated data by country.

```
1 covid_c = df.groupby(['Entity'])
```

Step 2

- If you see the result of the code below, you can see that the number of rows (data frame length) of the data frame for each key (which will be the country name in this case) is different. This means that the actual data provision status is different for each country. Remember that this is representative of one of the real-world situations.

```
# Separate the information of the created group by key and print them.
for key, group in covid_c:
    print('+key:', key)
    print('+number:', len(group))
    print(group. head())
    print('\n')
```

```
+key: ('Afghanistan',)
+number: 135
                Entity Code  total_vaccinations_per_hundred
Date
2021-02-22  Afghanistan  AFG                            0.00
2021-02-28  Afghanistan  AFG                            0.02
2021-03-16  Afghanistan  AFG                            0.13
2021-04-07  Afghanistan  AFG                            0.29
2021-04-22  Afghanistan  AFG                            0.58


+key: ('Africa',)
+number: 928
```

Step 2

- If you see the result of the code below, you can see that the number of rows (data frame length) of the data frame for each key (which will be the country name in this case) is different. This means that the actual data provision status is different for each country. Remember that this is representative of one of the real-world situations.

```
1 # Separate the information of the created group by key and print them.
2 for key, group in covid_c:
3     print('+key:', key)
4     print('+number:', len(group))
5     print(group. head())
6     print('\n')
```

Line 1, 3, 4, 5

- 1: Separate the information of the created group by key and print them.
- 3: Print the group's key name.
- 4: The number of data for the key (The number of data by country)
- 5: Print 5 lines for each group.

Step 2

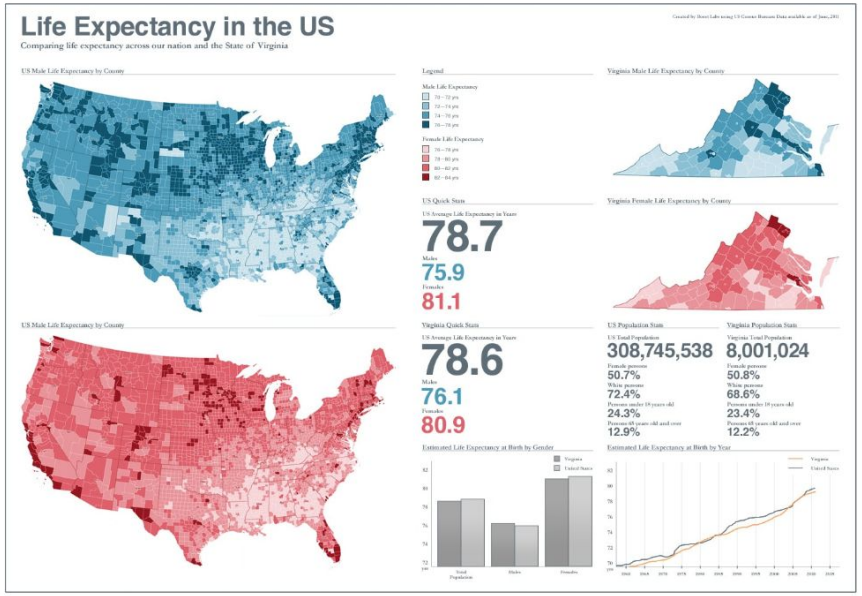
- Let's save the total for each group in a new data frame. We create statistics about the cumulative number of cases by 100 people in each country from the time of the corona outbreak to the present time.

```
1 total_df = covid_c[['total_vaccinations_per_hundred']].sum()
2 total_df.head()
```

Entity	total_vaccinations_per_hundred
Afghanistan	3241.50
Africa	27423.11
Albania	13692.69
Algeria	664.55
Andorra	6928.74

Step 3

- We have learned and practiced data visualization of charts for various descriptive statistics using matplotlib or seaborn library. However, in the field of data visualization, there are many cases of using map objects as shown in the image below. The result of this mini-project is to express the corona situation in the form of a data frame by changing the color according to the weight on the world map.



https://www.webdesignerdepot.com/2009/10/30-superb-examples-of-infographic-maps/

- Before solving the mission, we will learn how to express data on the map using the folium library.
- https://python-visualization.github.io/folium/

Step 3

1) Install folium library

- Move to the current virtual environment, and install the library through one of the two commands below.

  - **pip install folium**
  - **conda install folium -c conda-forge**: It is recommended to install using conda.

Step 3

2) Get coordinate information of the location you want

- You need to know the latitude and longitude information of the location to display a map of the location you want. The easiest way to get a latitude and longitude location is to use the Google Maps service. The method below is the method when using a normal PC.

  - Go to https://www.google.com/maps
  - Search the location you want on the map or move it by dragging the mouse.
  - Click twice the location where you want to accurately get latitude and longitude information.
  - When you click the location, you can copy the latitude and longitude information.

Step 3

2) Get coordinate information of the location you want

- The image below is the result of searching for the location of Samsung Electronics' headquarters in Korea as a sample.



Step 3

3) Create a map

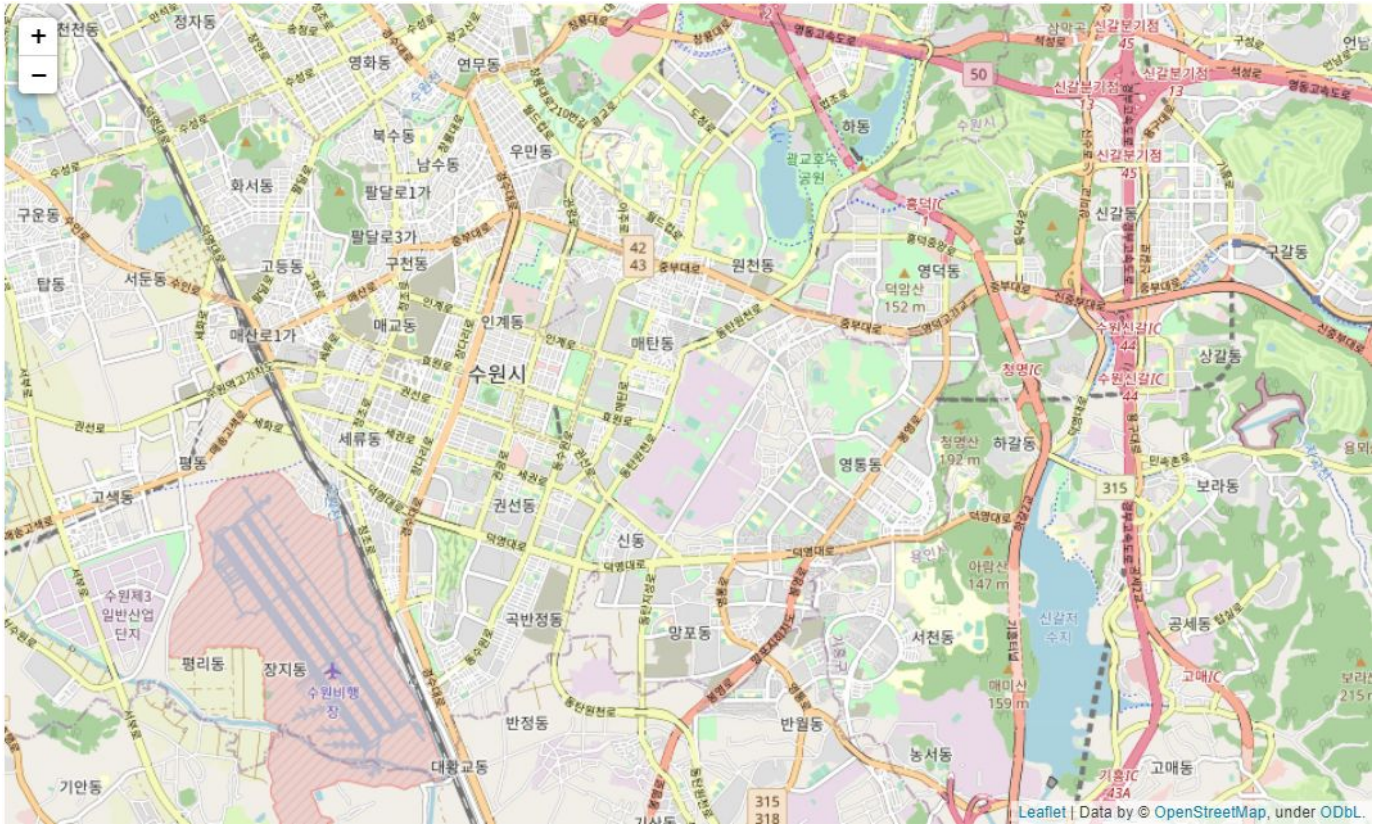
```
1  import folium
2
3  map = folium.Map(location = [37.2594750011864, 127.05145091394964],
4                   zoom_start=13,
5                  )
6
7
8
9  map
```

Line 3~4

- 3: Enter the coordinates of the center of the map in the order of latitude and longitude.
- 4: Magnification factor for the initial rendering of the map.

Step 3

3) Create a map



Step 3

4) Apply a style to the map

- You can specify the graphic style of the map displayed through the tiles parameter input of the map() function. If no value is entered, the openstreetmap style is applied. Here are a few of the most used ones:

  - tamenterrain
  - stamentoner
  - stamenwatercolor
  - cartodbpositron
  - cartodbdark_matter
  - openstreetmap

- There are several styles, you can change them one by one and see them for yourself.

Step 3

4) Apply a style to the map

```
1  import folium
2
3  map = folium.Map(location = [37.2594750011864, 127.05145091394964],
4                   zoom_start=13,
5                   tiles="stamenwatercolor",
6                  )
7
8  map
```

Line 3, 4, 5

- 3: Center coordinates of the map
- 4: Magnification factor for the initial rendering of the map
- 5: If nothing is specified, the default value is openstreetmap.

Step 3

4) Apply a style to the map



Step 3

5) How to display marker icons and information at specific locations

```
marker_map = folium.Map(location=[45.372, -121.6972], zoom_start=12, tiles="Stamen Terrain")

folium.Marker(
    location=[45.3288, -121.6625],
    popup="Mt. Hood Meadows",
    icon=folium.Icon(icon="cloud"),
).add_to(marker_map)

folium.Marker(
    location=[45.3311, -121.7113],
    popup="Timberline Lodge",
    icon=folium.Icon(color="green"),
). add_to(marker_map)

folium.CircleMarker(
    location=[45.3800, -121.6000],
    radius=100,
    popup="circle",
    color="#3186cc",
    fill=True,
    fill_color="#3186cc",
).add_to(marker_map)


marker_map
```

Step 3

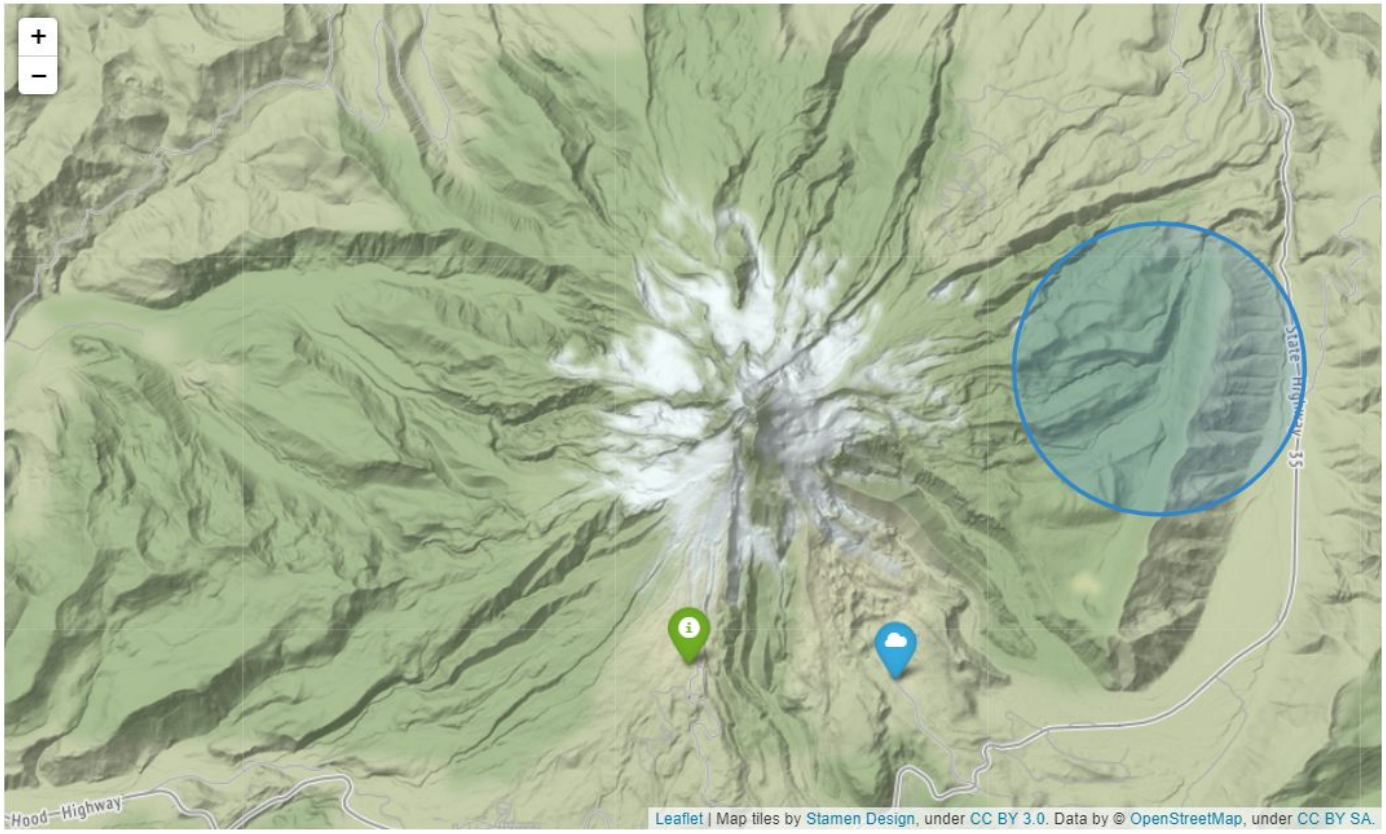
5) How to display marker icons and information at specific locations

Line 4, 5, 6, 15, 17, 18, 19, 20, 21

- 4: Latitude and longitude information where the marker will be displayed
- 5: Information to be displayed as a pop-up message when a marker is clicked
- 6: Marker icon style
- 15: A method to specify an area in a circle.
- 17: Specify the size of the circle
- 18: Information to be displayed as a pop-up message when the circle is clicked
- 19: Color information of the borderline of the circle
- 20: Decide whether to paint the inside of the circle
- 21: Color information to be painted in a circle

Step 3

5) How to display marker icons and information at specific locations



Step 3

6) How to display a step-by-step diagram in the map area

- It is a method to visualize data by coloring the area surrounded by a certain boundary on a map, such as a boundary line between specific countries or an administrative district.
- As the value of the information to be delivered increases, the color painted in the corresponding area becomes darker.

Step 3

6) How to display a step-by-step diagram in the map area

```
import pandas as pd

url = (
"https://raw.githubusercontent.com/python-visualization/folium/master/examples/data"
)
state_geo = f"{url}/us-states.json"
state_unemployment = f"{url}/US_Unemployment_Oct2012.csv"
state_data = pd.read_csv(state_unemployment)

m = folium.Map(location= [48, -102], zoom_start=3)

folium.Choropleth(
    geo_data=state_geo,
    name="choropleth",
    data=state_data,
    columns=["State", "Unemployment"],
    key_on="feature.id",
    fill_color="YlGn",
    fill_opacity=0.7,
    line_opacity=0.2,
    legend_name="Unemployment Rate (%)",
).add_to(m)

folium.LayerControl().add_to(m)

m
```

Step 3

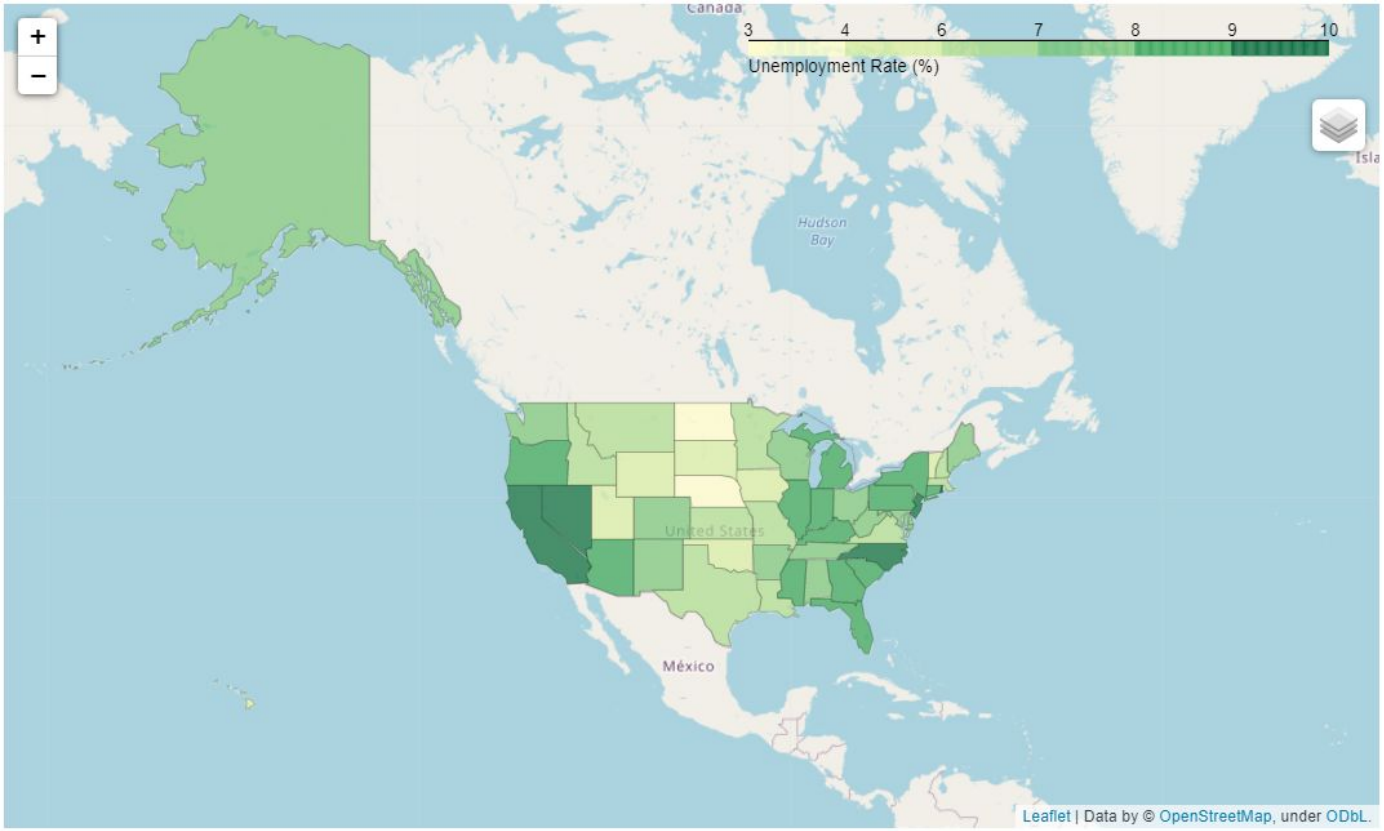
6) How to display a step-by-step diagram in the map area

Line 13, 15, 16, 17

- 13: geoJson data for the administrative district
- 15: Name of the data to be loaded into the administrative district
- 16: Column names and variables to use for this map in the dataframe
- 17: Matching between administrative districts in data and administrative districts in geojson

Step 3

6) How to display a step-by-step diagram in the map area



Step 4

- We can visualize the already processed data frame of the cumulative number of confirmed cases by 100 people in each country by applying the visualization method to the map learned in step 3.
- https://ourworldindata.org/covid-vaccination-global-projections

Step 4

```
import folium
import json

center = [35.762887375145795, 84.08313219586536]

m = folium.Map(location=center, zoom_start=2,
               max_bounds = True,
               min_zoom =1, min_lat =-84,
               max_lat = 84, min_lon =-175, max_Lon = 187,
              )

geo_path = "./data/world_geojson/World_Countries_Generalized.geojson"

json_data = json.load(open(geo_path, encoding='utf-8'))

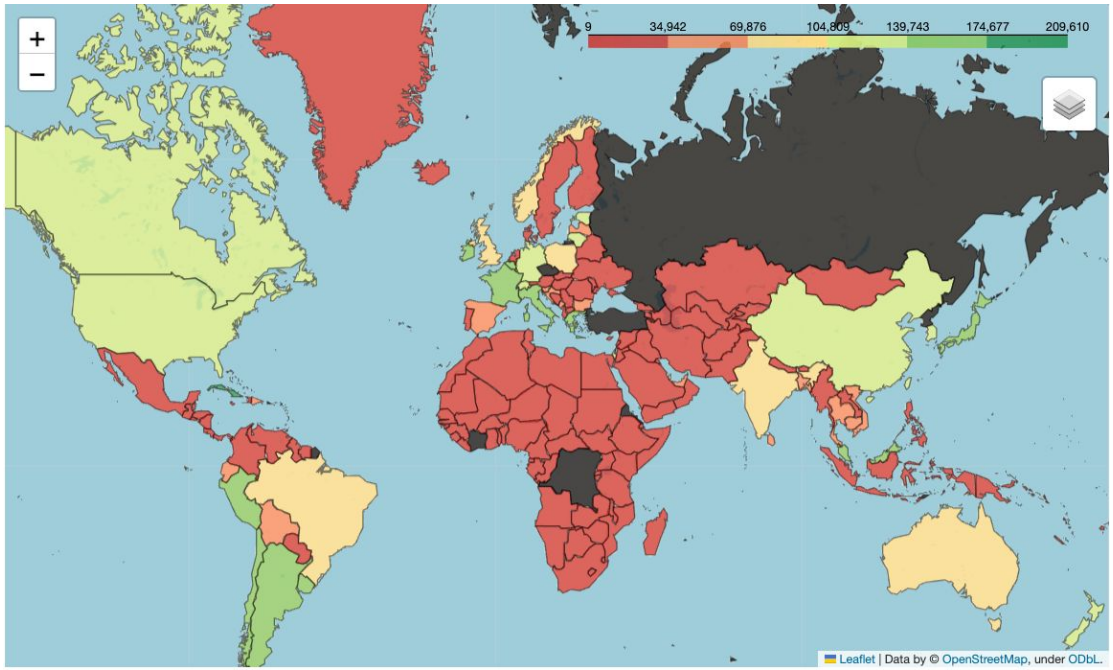
folium.Choropleth(geo_data = json_data,
                  data= total_df,
                  columns=(total_df.index, 'total_vaccinations_per_hundred'),
                  key_on = 'properties.COUNTRY',
                  fill_color = 'RdYlGn',
                  fill_opacity = 0.7,
                  line_opacity = 0.5,
                 ).add_to(m)


folium.LayerControl().add_to(m)
```

```
<folium.map.LayerControl at 0x103d2e920>
```

Step 4

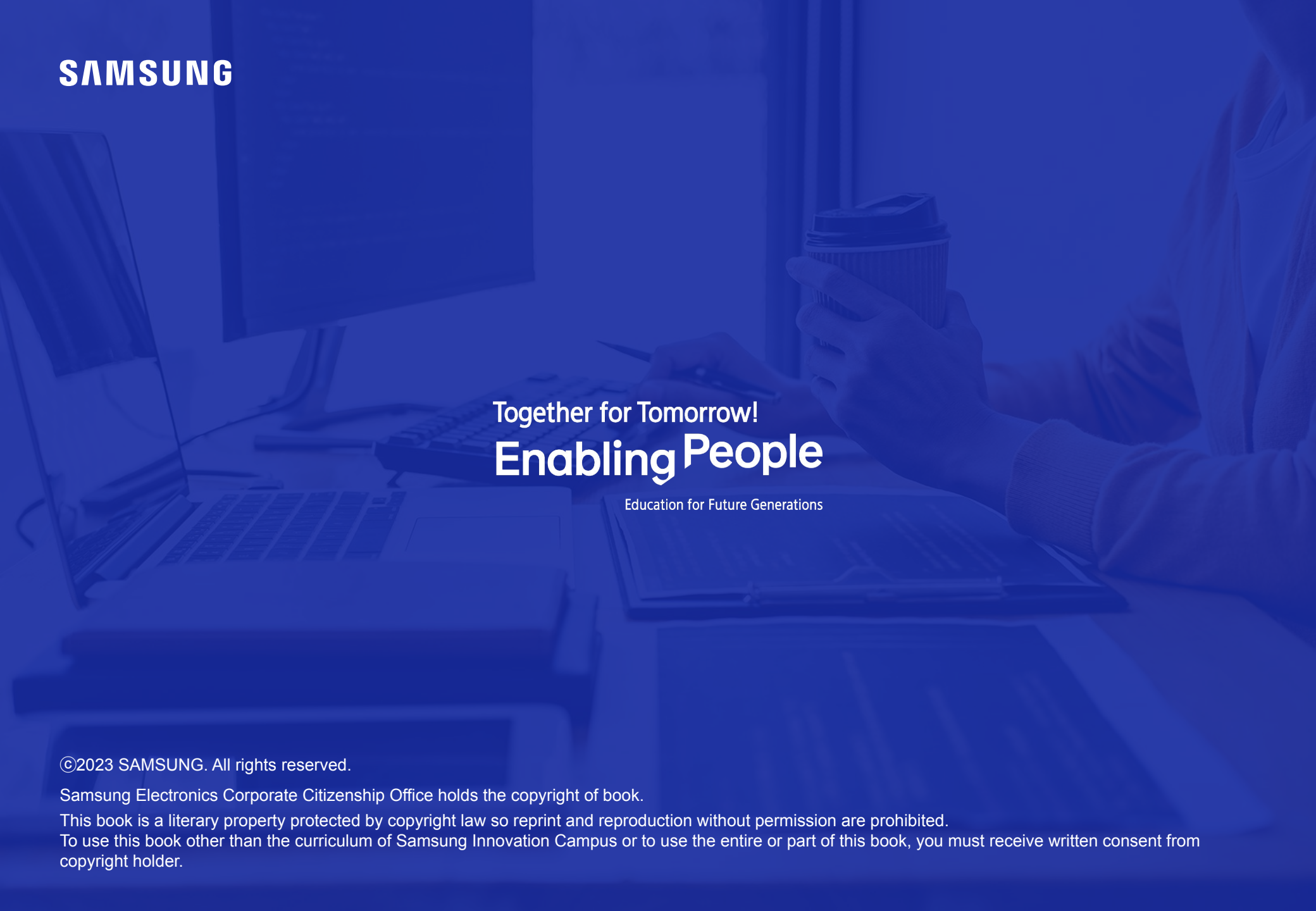
```
1 m
```



Line 1

- This is a visualization of the cumulative number of infected people by a group of 100 people from the time of the corona outbreak based on the current time.


End of
Document



ⓒ2023 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of book.

This book is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this book other than the curriculum of Samsung Innovation Campus or to use the entire or part of this book, you must receive written consent from copyright holder.